We Buy Clean Trash — Testing Guide

Phases 1–4 · Foundation → Auth → Resident flow → Admin core · Updated April 20, 2026

What you should be able to do so far

1. Sign up as a new **resident** and land on a working dashboard.
2. Promote yourself to **admin** from the CLI and access the admin console.
3. Send **invite links** that grant operator / depot / manager / admin roles.
4. Browse the resident experience end-to-end: onboarding, order bags (mock Stripe), scan a bag, use the payout calculator, view the recycling guide, redeem a gift card, edit your profile.
5. Run the admin core: KPI dashboard, residents list, staff invites (create + revoke), zones + depots CRUD, yellow-sheet pricing editor (with price history), redemption fulfillment queue.

> **Not built yet (skip).** Public landing page, pre-signup `/scan` (AI camera), zone auto-assignment at signup, and Operator / Depot / Manager flows (Phases 5–9).

0 · Prerequisites

Environment

1. `.env.local` has the Firebase web config (`NEXT_PUBLIC_FIREBASE_*`), the service account (`FIREBASE_ADMIN_SA`, one JSON line), and the storage bucket (`NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET`).
2. Firestore rules are in "default deny" — all writes go through API routes with the Admin SDK, so you don't need to deploy rules yet.
3. Firebase Auth providers enabled in the Firebase console: **Email/Password** (and Google if you want to test that path).

Install + seed

```
npm install
npm run seed:materials      # writes 7 materials docs with pilot defaults
npm run dev                 # boots http://localhost:3000
```

> **Tip.** After the dev server starts, keep a second terminal open for the CLI helpers listed below (`set-admin`, `dev:issue-bags`, `whoami`).

Useful CLI helpers

Command	Purpose
`npm run set-admin -- you@example.com`	Promote an existing user to admin (sets custom claim + updates `users` doc).

`npm run whoami -- you@example.com`	Dump UID, custom claims, and `users/{uid}` doc for debugging.
`npm run dev:issue-bags -- you@example.com`	Create a sticker sheet + 10 `bags` for a resident so scan-bag tests have something to scan.
`npm run seed:materials`	Re-seed yellow-sheet pricing (add `--force` to overwrite).

> **Claim refresh.** Anytime you change a user's role with the CLI, that user must **sign out and back in** before the new claim reaches the browser.

Phase 1 · Auth & role routing

1.1 — Resident self-signup

1. Go to `http://localhost:3000/signup`.
2. Fill in name, email, password, and a full address (street, city, state, ZIP).
3. Submit.

`EXPECT` redirected to `/resident/welcome` (the onboarding card). The user's custom claim is now `role: resident` and a{' '} `transactions/<id>` doc of type `signup_bonus` for **+10,000 pts** is written.

1.2 — Login

1. Sign out (top-right on `/resident`).
2. Go to `/login`, enter the same email/password.

`EXPECT` redirect to `/resident`.

1.3 — Role-guarded route

1. While signed in as the resident, try to visit `/admin`.

`EXPECT` redirect back to `/resident` (the `requireRole('admin')` guard bounces you to the home for your current role).

1.4 — Unauthenticated redirect

1. Sign out.
2. Visit `/resident`, `/admin`, or `/operator` directly.

`EXPECT` each one redirects to `/login`.

1.5 — Promote yourself to admin

1. In the second terminal, run `npm run set-admin -- you@example.com`.
2. Back in the browser, sign out and sign back in.
3. Visit `/admin`.

`EXPECT` the admin dashboard layout with the sidebar.

1.6 — Invite an operator (end-to-end)

1. As admin, go to `/admin/invites`.
2. Fill in an email you can access (or any; you'll visit the link manually), role **Operator**, leave zone unassigned.
3. Click **Create invite**.
4. The success box shows an invite URL of the form `http://localhost:3000/invite/<token>`. Copy it.
5. Open a **private window**, paste the URL, accept the invite with a new email/password.

`EXPECT` the new account is created with custom claim `role: operator`, the invite row in `/admin/invites` flips to **Accepted**, and visiting `/operator` in the private window shows the (stub) operator placeholder.

1.7 — Revoke a pending invite

1. Create another invite (don't accept it).
2. In the invites table click **Revoke**.

`EXPECT` row status changes to **Revoked**; opening the link now shows the "invite unavailable" state instead of the acceptance form.

Phase 3 · Resident flow

> **Before testing.** Sign in as the resident you created in Phase 1. For scan-bag, run `npm run dev:issue-bags -- <resident-email>` in the second terminal — it prints 10 bag codes like `BAG-4237-01` you can use in the manual-entry field.

3.1 — Onboarding card

1. If you just signed up, you land on `/resident/welcome`.
2. Read the three-step card and click **Let's go**.

`EXPECT` redirect to `/resident`, and the `users` doc now has `onboardingCompletedAt` set. Revisiting `/resident` never shows the welcome card again.

3.2 — Home dashboard

On `/resident` you should see:

- Greeting with your first name.
- Points **10,000** / value **$1.00** (the formula is `100,000 pts = $10`).
- "Order more bags" card at the top (primary revenue action).
- "No pickup scheduled yet" placeholder (routes land in Phase 5).
- "What's your trash worth?" link to the calculator.
- Bottom nav: Home / Scan / Bags / Rewards / Profile.

3.3 — Order bags (mock Stripe)

1. Open `/resident/order-bags`.
2. Use the stepper to set quantity to **1**.
3. Confirm the total shows **+$10 shipping** because the subtotal is under the $20 free-shipping threshold.
4. Bump quantity to **3** — shipping should switch to free.
5. Click **Pay (stub)**.

`EXPECT` redirect to `/resident/order-bags/success?order=...`. A new `bagOrders` doc exists in Firestore with `stripeStubSuccess: true` and `status: queued`. Route enqueue is deferred to Phase 5.

3.4 — Scan a bag (QR or manual)

1. Run `npm run dev:issue-bags -- you@example.com`; copy a `BAG-XXXX-01` code.
2. Visit `/resident/scan-bag`.
3. Use **manual entry**, paste the code, pick **Mixed** or **Separated**, then attach a photo (use the file picker on desktop or the camera on mobile).
4. Submit.

`EXPECT` a success toast. A new `pickups` doc is created with `status: pending` + `doorstepPhotoUrl` in Storage, and the matching `bags` doc flips to `pending_pickup`. Rescanning the same bag now errors with *bag already in use*.

> **HTTPS for camera.** `localhost` is treated as secure, so the live QR camera works on desktop Chrome. On a phone, the iOS/Android browser may block camera access unless you serve over HTTPS (use `ngrok` or the manual-entry fallback).

3.5 — Payout calculator

1. Visit `/resident/calculator`.
2. Enter weights for aluminum, cardboard, PET, etc.

`EXPECT` a live-updating dollar estimate. The page reads `materials` live — if you edit a price in `/admin/pricing`, the calculator reflects it on reload.

3.6 — Recycling guide

`/resident/guide` renders the good vs. not-accepted examples + the oil-can prep tip (static content for now).

3.7 — Rewards + redemption

1. Rewards gate at **100,000 pts = $10**. A fresh signup has only 10,000 so redemption will be blocked. To actually test it, manually add points via the Firebase console: set `users/{uid}.pointsBalance` to `100000` and add a matching `transactions` entry if you care about ledger cleanliness.
2. Visit `/resident/rewards` and click **Amazon $10**.

`EXPECT` balance drops by 100,000, a `redemptions` doc with `status: pending` is created, and a new `transactions` ledger entry of type `redemption` (negative delta) is written — all in one transaction. **Pay Trash Bill** and **Donate** are disabled stubs.

3.8 — Profile

`/resident/profile` shows the address saved at signup, pickup day (read-only, admin-assigned — will be blank for pilot until zones are assigned), and a Sign out button.

Phase 4 · Admin core

> **Before testing.** Sign in as the admin user you promoted in 1.5. Everything below is under `/admin/*`.

4.1 — Dashboard

1. Visit `/admin`.

`EXPECT` four KPI cards:

- **Active residents** — count of `users` where role=resident.
- **Bags issued this month** — count of `bags` created this calendar month (run `dev:issue-bags` to bump this).
- **Material in stock** — sum of all `inventory.weight` (0 lbs until Phase 7 processing runs).
- **Revenue · {current month}** — sum of non-cancelled `bagOrders.total` this month.

Below the KPIs: inventory by material (7 rows, zeros for now). Zone performance, contamination alerts, and operator leaderboard are Phase 9.

4.2 — Users list

1. Visit `/admin/users`.

`EXPECT` a table of every resident (name, email, zone, points, dollar value). Your signed-up resident from 1.1 appears with 10,000 pts / $1.00. Click "Manage staff invites →" to jump to invites.

4.3 — Create + list + revoke invites

1. Covered end-to-end in tests 1.6 and 1.7.
2. Spot-check: Role picker shows zone input only for **Operator**, and depot input only for **Depot worker** / **Depot manager**.
3. Expired invites appear with status **Expired** (TTL is 7 days from creation; you can force this by editing `invites/{token}.expiresAt` in the Firebase console to a past timestamp).

4.4 — Zones & depots CRUD

1. Visit `/admin/zones`.
2. Create a depot: *Name* "Kirk's Yard", *Street* "100 Pilot Ln", *City* "Springfield", *State* "OH", *ZIP* "45501".
3. Create a zone: *Name* "North Pilot", *Depot* Kirk's Yard, *Pickup day* Wednesday.
4. Try to **Delete** the depot — should error with "zones still reference this depot".
5. Delete the zone first, then the depot.

`EXPECT` the `zones` and `depots` docs are created/removed; the depot's `zoneIds` array is kept in sync (zone creation uses a Firestore transaction to set both).

4.5 — Yellow-sheet pricing editor

1. Visit `/admin/pricing`.
2. Edit a value — e.g. set Aluminum *Market $/lb* to **1.50**, leave payout at **30%**.
3. Watch the rightmost "Resident $/lb" column update live to $0.450.
4. Click **Save pricing**.

`EXPECT` all 7 `materials` docs updated and a new `priceHistory` snapshot written in the same Admin SDK transaction (check the Firebase console). Reload `/resident/calculator` as the resident — the new price is used.

4.6 — Redemption fulfillment queue

1. Trigger a redemption as the resident (see 3.7 for how to give them enough points).
2. As admin, visit `/admin/redemptions`.
3. The request shows up under **Pending** (resident name, brand, points, $, date).
4. Click **Mark fulfilled**. In the prompt, paste a fake code like `AMZN-TEST-1234` (or leave blank) and confirm.

`EXPECT` the row moves to **Recently fulfilled** with the code shown, and the `redemptions` doc now has `status: fulfilled`, `fulfilledBy: <admin uid>`, and `fulfilledAt` set.

Quick regression checklist

Five-minute smoke test before moving to Phase 5:

1. Fresh signup works and hits `/resident/welcome`.
2. Signed-out resident cannot see `/resident`; signed-in resident cannot see `/admin`.
3. Invite link accepted in an incognito window grants the correct role.
4. Resident order-bags flow lands on the success page and writes a `bagOrders` doc.
5. Scanning an issued bag flips it to `pending_pickup` and creates a `pickups` doc with a photo in Storage.

6. Saving new material prices writes a `priceHistory` doc (visible in the Firebase console).
7. A resident redemption surfaces in `/admin/redemptions` and can be marked fulfilled.
